

计算机科学与技术学院 大数据管理与分析 课程实验报告

实验题目：实验一 大数据系统基本实验

(Linux 与 Hadoop 基本操作/HDFS 常用操作/MapReduce 初级编程)

学号：202400390056

日期：2026.10.02

班级：2026 级 1 班

姓名：于昕杨

实验代码仓库 (GitHub, 公开) : <https://github.com/YXY9527797/lab1-bigdata>

一、实验目的

1. 熟悉 Linux 常用命令 (目录切换、文件与目录的增删改查、内容查看、查找、打包压缩、权限修改等), 为后续实验打基础。
2. 掌握 Hadoop 的启动方式, 理解 HDFS 在 Hadoop 体系结构中的角色, 熟练使用 HDFS 常用 Shell 命令与 Java API。
3. 掌握基本的 MapReduce 编程方法, 能够用 MapReduce 解决数据去重、数据排序、信息挖掘等常见数据处理问题。
4. 掌握把实验代码整理并托管到 GitHub 公开仓库的工程实践。

二、实验软件和硬件环境

项目	配置
硬件	Intel x86_64 虚拟机, 4 核 CPU, 内存 7.7 GB, 磁盘 24 GB (可用约 4.5 GB)
操作系统	Ubuntu 22.04.5 LTS, Linux 内核 6.8.0-40-generic
JDK	OpenJDK 11.0.32 (Hadoop 3.3.6 官方支持 JDK 8/11)
Hadoop	Apache Hadoop 3.3.6, 伪分布式部署 (NameNode / DataNode / ResourceManager / NodeManager 同机)
HDFS 地址	hdfs://localhost:9000 (NameNode Web UI: http://localhost:9870; YARN Web UI: http://localhost:8088)
其他工具	bash 5.1、GNU coreutils、tar/grep/find、LibreOffice 7.3 (报告排版)、git 2.34

三、实验原理和方法

3.1 Linux 常用操作

Linux 一切皆文件, 日常操作围绕“目录树”展开: `cd`/`ls`/`mkdir`/`rmdir`/`cp`/`mv`/`rm` 负责目录与文件的组织; `cat`/`tac`/`more`/`head`/`tail`/`grep` 负责文本查看与检索; `touch`/`chown` 负责时间戳与属主权限; `find` 负责按名字搜索; `tar` 负责打包压缩。Linux 的权限模型为“属主/属组/其他人”各三位 (rwx), 因此写 `/`、`/usr` 这类系统目录以及执行 `chown root` 都需要 root 身份。

3.2 HDFS 与 Hadoop 基本操作

HDFS 采用 Master/Slave 架构：NameNode 保存整个文件系统的命名空间与元数据（文件—数据块的映射、权限、副本位置），DataNode 保存真实的数据块。客户端读写时先访问 NameNode 取得元数据，再直接与 DataNode 交换数据流；YARN 的 ResourceManager/NodeManager 负责为 MapReduce 作业分配容器资源。HDFS 提供了两套等价接口：命令行 `hdfs dfs ...` 与 Java API (`org.apache.hadoop.fs.FileSystem`)，后者是前者的底层实现。

3.3 MapReduce 编程模型

MapReduce 把计算抽象为 Map（把输入解析成 `<key, value>`）与 Reduce（对相同 key 的 value 集合做归并）两个阶段，框架在两者之间自动完成 分区、排序、合并、洗牌（Shuffle）。本实验三个题目正是利用“框架会对 key 排序、相同 key 会进入同一次 reduce 调用”这两个特性来实现：

- 合并去重：以“整行内容”为 key，reduce 中每个 key 只输出一次，重复行自然消失，输出同时按行有序。
- 排序并编号：以整数本身为 key，框架排序后 reduce 收到升序 key；再用 1 个 Reduce 任务 + 计数器即可产生全局连续位次。
- 祖孙关系挖掘：一趟 MapReduce 完成自连接——每行 (child, parent) 发出两条记录，以“中间人”为 key 聚齐其子辈与父辈，reduce 中做笛卡尔积得到 (孙辈, 祖辈)。

四、实验步骤

全部代码、输入数据、运行日志与报告源码均已放入公开仓库 `lab1-bigdata/`，目录结构如下：

lab1-bigdata/	
├─ env/	Hadoop 单节点环境：配置模板、集群启停脚本、可写根视图脚本
├─ part1-linux-hadoop/	第 1 部分：Linux 常用操作 + Hadoop(HDFS) 基本操作
├─ part2-hdfs/	第 2 部分：HDFS 常用操作 (Shell 实现 + Java API 实现)
├─ part3-mapreduce/	第 3 部分：三个 MapReduce 作业 + 输入数据 + 标准答案
├─ report/	本实验报告 (HTML 源 + 生成 PDF/DOCX 的脚本)
└─ docs/run-logs/	各步骤真实运行日志 (共 1745 行)

4.1 第 1 部分：Linux 常用操作（17 道题全部真实执行）

脚本 `part1-linux-hadoop/linux_basics.sh` 依次执行教材中的 17 道题，每一步都回显命令并打印真实输出（日志 `logs/linux-basics.log`，808 行）。为便于对照，命令与执行结果摘要如下：

序	命令	执行内容	执行结果摘要
1	cd	cd /usr/local; cd ..; cd ~	依次进入 /usr/local、/usr、/home/<用户> 列出 /usr 下
2	ls	ls -l /usr	bin、include、lib、local、sbin、share、src 等

3	mkdir	mkdir /tmp/a; mkdir -p /tmp/a1/a2/a3/a4	目录创建成功，find 确认四级目录均生成
4	rmdir	rmdir /tmp/a; rmdir -p /tmp/a1/a2/a3/a4	空目录删除成功；删到 /tmp 时报“设备或资源忙”（见问答题 1）
5	cp	cp ~/.bashrc /usr/bashrc1; mkdir /tmp/test; cp -r /tmp/test /usr	文件与目录复制成功，/usr/bashrc1 与 /usr/test 均存在
6	mv	mv /usr/bashrc1 /usr/test; mv /usr/test /usr/test2	移动与重命名成功
7	rm	rm /usr/test2/bashrc1; rm -r /usr/test2	文件与目录均删除成功，再次 ls 提示“没有那个文件或目录”
8	cat	cat ~/.bashrc	完整输出 121 行 .bashrc 内容
9	tac	tac ~/.bashrc	按行号逆序输出同一文件
10	more	more -10 ~/.bashrc	分页查看（脚本中以 -10 限定页长以便观察）
11	head	head -n 20 ~/.bashrc; head -n -50 ~/.bashrc	分别输出前 20 行、去掉末尾 50 行后的全部行
12	tail	tail -n 20 ~/.bashrc; tail -n +51 ~/.bashrc	分别输出最后 20 行、第 51 行起的全部行
13	touch	touch /tmp/hello; touch -d "5 days ago" /tmp/hello	创建空文件，stat 显示修改时间被回拨 5 天
14	chown	chown root /tmp/hello	属主由普通用户变为 root:root
15	find	find ~ -name .bashrc	找到 /home/<用户>/.bashrc
16	tar	mkdir /test; tar -czf /test.tar.gz test; tar -xzf /test.tar.gz -C /tmp	在根目录打成 150 字节压缩包，并成功解压到 /tmp
17	grep	grep -n 'examples' ~/.bashrc	命中第 2、3、102 行（退出码 0）

说明：本机当前账户没有 root 权限，而第 5~7、14、16 题要求写 `/usr`、`/` 或以 root 身份改属主。为此仓库提供了 `env/lab-root.sh`：利用 Linux 用户命名空间取得映射 root 身份，再用 `bind mount + chroot` 构造一个“可写的根视图”（`/usr`、`/` 可写，`/usr/bin`、`/usr/local/hadoop` 等真实内容原样可见，`/tmp` 也是挂载点），因此上述命令路径与教材完全一致且全部真实执行成功，而所有写操作只落在临时视图内，真实系统毫发无损。有 root/sudo 的机器上直接 `bash linux_basics.sh` 即可。

4.2 第 1 部分（续）：Hadoop 基本操作

脚本 `part1-linux-hadoop/hadoop_basics.sh` 完成教材最后 4 个小题（日志 `logs/hadoop-basics.log`，163 行）。

① 启动 Hadoop 并为 hadoop 用户创建 HDFS 用户目录。启动后 jps 显示 4 个守护进程：

```
$ jps -l
11  org.apache.hadoop.hdfs.server.namenode.NameNode
116 org.apache.hadoop.hdfs.server.datanode.DataNode
117 org.apache.hadoop.yarn.server.resourcemanager.ResourceManager
475 org.apache.hadoop.yarn.server.nodemanager.NodeManager
$ hdfs dfsadmin -report | grep -A2 "Live datanodes"
Live datanodes (1):      Name: 127.0.0.1:9866 (localhost)
$ hdfs dfs -mkdir -p /user/hadoop
$ hdfs dfs -chown hadoop:supergroup /user/hadoop
$ hdfs dfs -ls -d /user/hadoop
drwxr-xr-x  - hadoop supergroup          0 2026-10-02 00:17 /user/hadoop
```

② 在 /user/hadoop 下创建 test 文件夹并查看。

```
$ hdfs dfs -mkdir -p /user/hadoop/test
$ hdfs dfs -ls -R /user/hadoop
drwxr-xr-x  - yxy supergroup          0 2026-10-02 00:17 /user/hadoop/test
```

③ 把本地 ~/.bashrc 上传到 HDFS 的 test 文件夹并查看。上传后用 MD5 校验两端内容一致：

```
$ hdfs dfs -put -f ~/.bashrc /user/hadoop/test
$ hdfs dfs -ls /user/hadoop/test
-rw-r--r--  1 yxy supergroup      3968 2026-10-02 00:18 /user/hadoop/test/.bashrc
本地  : dacd0a2d71506e39140b00bcd3007eb
HDFS  : dacd0a2d71506e39140b00bcd3007eb
```

④ 把 HDFS 上的 test 文件夹复制回本地 /usr/local/hadoop。取回后同样以 MD5 校验通过：

```
$ hdfs dfs -get /user/hadoop/test /usr/local/hadoop
$ ls -la /usr/local/hadoop/test
-rw-r--r-- 1 root root 3968 10月  2 00:18 .bashrc
HDFS  : dacd0a2d71506e39140b00bcd3007eb
本地  : dacd0a2d71506e39140b00bcd3007eb
```

4.3 第 2 部分：HDFS 常用操作（Shell 与 Java API 各实现一遍）

题目要求“编程实现以下功能，并利用 Hadoop 提供的 Shell 命令完成相同任务”。因此本实验把 10 道题封装成同一套子命令，写了两个实现，再用同一个测试场景脚本（part2-hdfs/test-sequence.sh）分别驱动，输出可直接逐条对照：

- Shell 实现：hdfs_shell.sh（日志 logs/hdfs-shell.log，199 行）
- Java 实现：java/src/main/java/edu/sdu/bigdata/hdfs/*.java（10 个任务类 + 工具类 + 统一 CLI，日志 logs/hdfs-java.log，212 行）

题	功能要求	Shell 命令实现	Java API 实现
	上传文件；已存在	-test -e 判断存在性； -	fs.exists; fs.append/
1	时由用户指定追加或覆盖	appendToFile 追加/ -put -f 覆盖	fs.copyFromLocalFile(false,true,...)
2	下载；本地同名文件自动重命名	循环探测 name.1、name.2 后 -get	同上探测后用 fs.copyToLocalFile

3	把文件内容输出到终端	<code>hdfs dfs -cat</code>	<code>fs.open + IOUtils.copyBytes</code>
4	显示权限/大小/创建时间/路径	<code>-ls</code> 与 <code>-stat '%a %b %y %n'</code>	<code>fs.getFileStatus</code> → <code>getPermission/getLen/getModificationTime/getPath</code>
5	递归列出目录下所有文件的信息	<code>hdfs dfs -ls -R</code>	自己写深度优先递归 (<code>listStatus</code>)
6	文件创建/删除; 父目录不存在则自动创建	<code>-mkdir -p + -touchz/-rm</code>	<code>fs.mkdirs + fs.create/fs.delete</code>
7	目录创建/删除; 空目录才删, 非空不删	<code>-mkdir -p; -count</code> 判空后 <code>-rmdir</code>	<code>fs.mkdirs; listStatus().length==0</code> 后 <code>fs.delete(p, false)</code>
8	追加内容到文件开头或结尾 (用户指定)	末尾: <code>-appendToFile</code> ; 开头: 取回→重组→ <code>-rm</code> → <code>-put</code>	末尾: <code>fs.append</code> ; 开头: 读出后 <code>fs.delete+fs.create</code> 重写
9	删除指定文件	<code>hdfs dfs -rm</code>	<code>fs.delete(path, false)</code>
10	把文件从源路径移动到目的路径	<code>-mkdir -p</code> 目的目录 + <code>-mv</code>	<code>fs.mkdirs + fs.rename</code>

两个实现的运行结果完全一致, 全部 10 道题均执行成功。关键结果摘录:

【题目 1-③】目标文件已存在且未指定模式 → 由用户交互指定 (输入 1 = 追加)
 目标文件 /user/yxy/hdfs-lab-java/upload.txt 已存在, 请选择处理方式:
 [1] 追加到原有文件末尾 (append)
 [2] 覆盖原有的文件 (overwrite)
 请输入 1 或 2: 已把 .../more.txt 的内容【追加】到 /user/yxy/hdfs-lab-java/upload.txt 末尾

【题目 2-②】本地已有同名文件 → 自动重命名
 本地已存在同名文件 upload.txt, 自动重命名为: upload.txt.1
 已下载到本地: .../local-java/download/upload.txt.1 (120 字节)

【题目 5】目录递归属性 (节选)

权限	属主	大小(字节)	创建/修改时间	路径
rw-r--r--	yxy	198	2026-10-02 00:21:38	hdfs://localhost:9000/user/yxy/hdfs-lab-java/upload.txt

共 8 个条目, 其中文件 4 个。

【题目 7-④】删除非空目录 (按要求不删除)
 目录 /user/yxy/hdfs-lab-java/nonempty 非空 (含 1 个子项), 按要求【不删除】:

【题目 8-②】追加内容到文件【开头】
 追加前内容 (120 字符): ... (略)
 追加位置: 文件开头 (HDFS 的 append 只能追加到末尾,

因此这里读出原内容后删除文件，再按“新内容+原内容”重建)
追加后内容 (168 字符) : >>> 这一行追加在文件开头 <<< ...

4.4 第 3 部分：MapReduce 初级编程

三个作业均提交到 YARN 以容器方式运行（脚本 `part3-mapreduce/run.sh`，日志 `logs/mr.log`，346 行），作业结束后自动与 `expected/` 下的标准答案逐行比对。

题目 1 文件的合并和去重。输入为 A.txt（6 行）与 B.txt（5 行），以整行内容为 key、`NullWritable` 为 value，reduce 中每个 key 只写出一次。作业 `job_1790871965624_0002` 运行成功（2 个 Map、1 个 Reduce），Reduce 输入 11 条、输出 9 条，正好去掉 2 条重复：

```
Job job_1790871965624_0002 completed successfully
Reduce input groups=9    Reduce input records=11    Reduce output records=9
运行结果 (part-r-00000) :                与 expected/dedup.txt 比对:
20170101 x      20170104 y                【一致】输出与标准答案完全相同
20170101 y      20170104 z
20170102 y      20170105 y
20170103 x      20170105 z
                20170106 x
```

题目 2 对输入文件的排序。三个输入文件共 11 个整数。以整数为 key 交给框架排序，Reduce 任务数设为 1，用计数器依次产生位次。作业 `job_1790871965624_0003` 运行成功：

```
Job job_1790871965624_0003 completed successfully
Map input records=11    Reduce input groups=11    Reduce output records=11
运行结果 (part-r-00000) :                与 expected/sort.txt 比对:
1  1      5 16      9 39      【一致】输出与标准答案完全相同
2  4      6 25     10 40
3  5      7 33     11 45
4 12      8 37
```

题目 3 对指定的表格进行信息挖掘。对 child-parent 表（14 行）做一趟自连接：每行发出 `key=parent, value="1"+child` 与 `key=child, value="2"+parent` 两条记录，reduce 中对“子辈集合 × 父辈集合”求笛卡尔积。作业 `job_1790871965624_0004` 运行成功，Map 输出 28 条、Reduce 输出 12 条祖孙关系：

```
Job job_1790871965624_0004 completed successfully
Map input records=14    Map output records=28    Reduce output records=12
运行结果 (part-r-00000) :                与 expected/grandparent.txt 比对:
Mark  Jesse      Steven Frank                【一致】输出与标准答案完全相同
Mark  Alice      Steven Mary
Philip Jesse      Jone   Frank
Philip Alice      Jone   Mary
Jone   Jesse      Steven Alice
Jone   Alice      Steven Jesse
```

说明：MapReduce 按 key（即中间一代人）排序输出，因此结果的分组顺序与题目样例不同，但 12 组（孙辈，祖辈）关系与样例完全一致（已用 `diff` 校验）。

五、结论分析与体会

1. 三类操作的关系弄清楚了。Linux 命令负责“操作系统层面”的资源管理，Hadoop Shell 负责“集群层面”的操作，而 Java API 与 Shell 命令本质是同一套 HDFS 客户端的不同封装——实验中 Shell 与 Java 两个实现跑同一场景，输出逐条一致，这一点体会最深。
2. MapReduce 的核心是“把问题翻译成 key”。三个题目代码量都不大，难点在 key 的设计：以整行为 key 得到去重，以整数为 key 借框架排序，以中间人为 key 完成自连接。理解了“框架对 key 排序、相同 key 进同一次 reduce”这两条，就能把很多看似不同的问题统一处理。
3. Shell 与 Java API 并非完全等价，踩到了 3 个差异：① `hdfs dfs -put` 不会自动创建父目录，而 Java 的 `copyFromLocalFile` 内部会 `makedirs`；② `hdfs dfs -count` 输出的“目录数”包含目录自身，用 0 判空会误判，而 Java 的 `listStatus().length==0` 没有这个坑；③ HDFS 的 `append` 只能追加到末尾，Java API 里直接调 `fs.append`，Shell 里用 `-appendToFile`，两者都无法“追加到开头”，必须整体重写。这些差异提醒我：写代码时不能想当然地认为命令行与 API 行为一致，要以实测为准。
4. 环境问题往往比算法更费时间。本实验仓库路径中包含中文（“文档”），最初提交 MapReduce 作业时容器直接启动失败。通过查看 `container-launch` 诊断日志，发现路径里的中文被 JVM 转成了 `??????`，容器因此无法创建日志目录；最终在 `yarn-site.xml` 的环境变量白名单中加入 `LANG/LC_ALL/LANGUAGE`，并在 `mapred-site.xml` 里为容器 JVM 显式指定 `-Dfile.encoding=UTF-8 -Dsun.jnu.encoding=UTF-8` 后问题解决。过程虽然曲折，但让我真正理解了 YARN 容器环境是“被裁剪过的”，以及 locale/编码对分布式程序的影响。
5. 工程习惯上的收获。把每一步都落盘成日志、把标准答案放进 `expected/` 并用 `diff` 自动校验、把所有实验目录、配置与启动脚本都脚本化，使整个实验可以一键复现；代码按题目拆分成独立类并配统一 CLI，也便于后续实验复用。

六、问答题（就实验中遇到的问题自拟）

问 1：执行 `rmdir -p /tmp/a1/a2/a3/a4` 时，为什么最后会报“删除目录 '/tmp' 失败：设备或资源忙”？

答：`rmdir -p` 的语义不只是删除给定目录，而是“删除该目录，并继续向上删除已经变空的各级父目录”。本例中 `a4→a3→a2→a1` 被依次删除后，`/tmp` 恰好变成空目录，于是 `rmdir` 继续尝试删除 `/tmp` 本身；但真实 Linux 上 `/tmp` 是一个挂载点（通常由 `systemd` 挂成 `tmpfs`），而挂载点被内核视为“正在使用”，删除会返回 `EBUSY`，所以 `rmdir` 报“设备或资源忙”并停止，返回非 0 状态码。这不是配置错误，而是符合预期的保护行为：`/` 也会被同样地拒绝删除。实验也验证了这一点——把 `/tmp` 也做成挂载点后，各题行为与真实系统完全一致；反之若 `/tmp` 只是普通目录，`rmdir -p` 会把它一起删掉，从而连累后面所有涉及 `/tmp` 的题目。

问 2：题目 8 要求“追加内容到原有文件的开头”，为什么不能直接调用 HDFS 的 `append`？

最后是怎么实现的？

答：HDFS 的文件在存储层面由一系列**数据块**顺序组成，客户端写入时会先向 NameNode 申请租约（lease）并追加到**最后一个数据块**的末尾，因此 `FileSystem.append()` 与 `hdfs dfs -appendToFile` 在设计上就只支持“末尾追加”，HDFS 没有提供“在文件头部插入”的接口——这既符合 HDFS “一次写入、多次读取”的定位，也避免了插入导致所有块编号与副本位置重排的巨大开销。实验中只能采取“**整体重写**”的思路：先把原文件内容完整读出，再删除原文件、重新创建，并按“新内容 + 原内容”的顺序写入（Java 实现为 `fs.open` 读出 → `fs.delete` → `fs.create` 写出；Shell 实现为 `-get` → 本地重组 → `-rm` → `-put`）。实测追加前 120 字符、追加后 168 字符，内容与顺序均正确。需要注意的是这种做法对超大文件代价很高，实际生产中若要频繁在头部插入，应改用 HBase 等适合随机写的存储。

问 3：提交 MapReduce 作业时容器报 “Exception from container-launch” 并失败，如何定位和解决？

答：定位过程分三步：① 客户端只报 “Failing the application”，信息不足，于是到 YARN 的作业页面与 NodeManager 日志中查看 `container-launch` 的具体诊断；② 诊断日志里出现 `log4j:ERROR setFile(null,true) call failed`，路径被打印成 `/home/yxy/??????/deepseek-harness/...`，看到 6 个问号立刻意识到是**中文目录名“文档”无法编码**；③ 进一步确认原因是容器 JVM 的 locale 为 ASCII（容器环境变量经过白名单裁剪，`LANG` 没有传进去），JVM 用 ASCII 解析文件名时把不可表示的字符替换成 `?`，导致容器创建日志目录失败、启动脚本异常退出。解决办法是在 `yarn-site.xml` 的 `yarn.nodemanager.env-whitelist` 中补上 `LANG,LC_ALL,LANGUAGE`（让容器继承 UTF-8 的 locale），并在 `mapred-site.xml` 中用 `yarn.app.mapreduce.am.command-opts`、`mapreduce.map.java.opts`、`mapreduce.reduce.java.opts` 为容器 JVM 显式加上 `-Dfile.encoding=UTF-8 -Dsun.jnu.encoding=UTF-8`。重启集群后三个作业全部一次通过。这个问题的教训是：**把工程放在非 ASCII 路径下、以及容器环境变量被裁剪，都会引发很隐蔽的故障**，排查时应优先看守护进程与容器的原始日志，而不是只看客户端的报错。

七、实验代码（GitHub 仓库）

本实验全部代码、输入数据、运行日志与报告源码已整理并上传至公开仓库：

<https://github.com/YXY9527797/lab1-bigdata>

仓库内含：Hadoop 单节点环境配置与一键启停脚本、Linux/HDFS/MapReduce 全部源码（15 个 Java 类 + 若干 Shell 脚本）、实验输入数据与标准答案、各步骤真实运行日志，以及本报告的 HTML 源码与生成 PDF/DOCX 的脚本。